

PROMPT ENGINEERING MASTERY BUCKET · MODULE 02

Prompt Engineering Basics

The building blocks of every good prompt — structure, instructions, context, constraints, delimiters, roles, personas, output formatting, and reusable templates.

DIFFICULTY LEVEL

Beginner

ESTIMATED READING TIME

50-65 minutes

ESTIMATED PRACTICE TIME

80-100 minutes

PREREQUISITES

Module 01 — AI & LLM Fundamentals

LEARNING OUTCOMES

By the end of this module, you will be able to: structure a prompt with clear instructions, context, and constraints; use delimiters to separate prompt sections cleanly; apply role and persona prompting to shape tone and expertise; specify exact output formats; and build reusable prompt templates for repeated tasks.

Table of Contents

1.	Introduction
2.	Before You Begin
3.	Main Lesson
3.1	Prompt Structure
3.2	Instructions
3.3	Context
3.4	Constraints
3.5	Delimiters
3.6	Role Prompting
3.7	Persona Prompting
3.8	Output Formatting
3.9	Prompt Templates
4.	Visual Learning
5.	Real World Applications
6.	Practical Examples
7.	Code Examples
8.	Common Mistakes
9.	Best Practices
10.	Hands-on Activities
11.	Mini Project
12.	Review Questions
13.	Cheat Sheet
14.	Key Takeaways
15.	Additional Resources

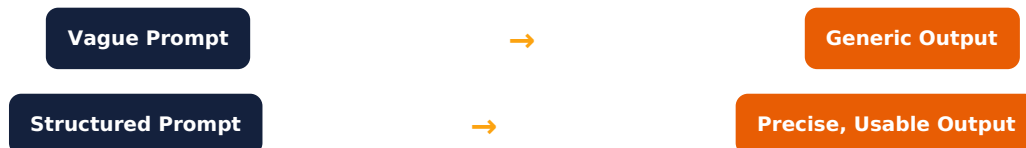
1. Introduction

What is this topic?

Module 01 taught you what's happening inside the model. This module teaches you what to put into it. Prompt engineering basics are the raw building blocks — instructions, context, constraints, structure — that every technique in this whole course (zero-shot, chain-of-thought, ReAct, agents) is assembled from.

Why is it important?

Most "the AI gave a bad answer" problems are actually prompt problems, not model problems. A vague prompt gets a vague answer. A well-structured prompt — with clear instructions, the right context, and explicit constraints — gets a dramatically better one, even from the exact same model.



Why should AI Engineers learn it?

These basics are the vocabulary you'll use to describe and debug every prompt you write from now on. When a prompt fails, an experienced engineer doesn't just "try again" — they diagnose it: was the instruction unclear? Was context missing? Were constraints unstated? This module gives you that diagnostic vocabulary.

Where is it used in the real world?

Every production AI feature — from a customer support bot's system prompt to a coding assistant's internal instructions to Claude Code's tool-use prompts — is built from these same core pieces, just combined and scaled up.

Why is it relevant today?

As more companies ship AI features, "prompt engineer" and "AI engineer" roles increasingly expect fluency in these basics as table stakes — the same way "knowing variables and loops" is table stakes before learning software architecture.

Note

This module focuses on the pieces of a prompt. Module 03 (Prompting Techniques) builds on these pieces to teach patterns like few-shot examples and chain-of-thought reasoning.

2. Before You Begin

Make sure you're comfortable with these two ideas from Module 01 before continuing:

RECAP

Tokens & context window — every part of your prompt (instructions, context, examples) consumes tokens from your fixed context window. Longer isn't automatically better.

RECAP

System vs. user messages — most APIs let you separate a "system prompt" (persistent instructions/persona) from the "user message" (the specific request). This module leans heavily on that split.



Tip

If you haven't made a real API call yet, revisit Module 01, Section 7 (Code Examples) — you'll want a working sandbox to try the exercises in this module.

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

3.1 Prompt Structure

DEFINITION

Prompt structure is the overall organization of a prompt — the order and grouping of instructions, context, examples, and constraints so the model can parse your intent clearly.

SIMPLE EXPLANATION

A well-structured prompt typically flows: role/persona (optional) → context → instructions → constraints → output format → the actual task. Unstructured prompts bury the real ask inside a wall of text.



💡 Like a well-organized work request email — subject line (task), background (context), specific asks (instructions), deadline/limits (constraints) — versus a rambling stream-of-consciousness message.

EVERYDAY EXAMPLE

Compare "write about dogs" (no structure) to "You are a vet. Given this dog's symptoms [context], write 3 possible causes [instruction], keep it under 100 words [constraint], as a bulleted list [format]."

WHY IT MATTERS

Structure reduces ambiguity, which reduces the chance the model guesses wrong about what you actually want.

COMMON MISTAKES

Mixing instructions and context together with no clear separation. Putting the actual task/question buried in the middle of a long paragraph.

BEST PRACTICES

Put the task instruction clearly, ideally near the start or end. Use delimiters (Section 3.5) to visually separate sections.

SUMMARY

Structure is about organizing a prompt's pieces predictably so the model — and future you — can immediately tell what's being asked.

3.2 Instructions

DEFINITION

Instructions are the explicit statement of what you want the model to do — the action itself.

SIMPLE EXPLANATION

Good instructions are specific, direct, and unambiguous. They tell the model the verb (summarize, translate, classify, extract) and the object (what exactly to act on).

💡 Like giving directions to a new employee — "handle the emails" is vague; "reply to every refund request with the standard refund template within 24 hours" is an instruction they can execute without guessing.

EVERYDAY EXAMPLE

Weak: "Tell me about this contract." Strong: "Summarize the payment terms in this contract in 3 bullet points."

WHY IT MATTERS

Vague instructions force the model to guess your intent — and it may guess wrong, especially on tasks with multiple valid interpretations.

COMMON MISTAKES

Bundling multiple unrelated instructions into a single vague sentence. Using soft language ("maybe," "if possible") for things you actually require.

BEST PRACTICES

Use direct, imperative language. Separate multiple instructions into a numbered list when there's more than one.

SUMMARY

Instructions are the action the model should take — the clearer and more specific, the less room for misinterpretation.

3.3 Context

DEFINITION

Context is the background information the model needs to complete the instruction correctly — facts, documents, prior conversation, or situational details.

SIMPLE EXPLANATION

Without context, the model can only rely on its general training knowledge, which may not match your specific situation, company, or data.

 *Like briefing a substitute teacher before class — without knowing which chapter the class is on, they can't teach the right lesson, even if they're a great teacher in general.*

EVERYDAY EXAMPLE

"Summarize this" (no context) versus pasting the actual document text before asking for a summary — the second version gives the model something concrete to work from.

WHY IT MATTERS

Missing context is one of the top causes of hallucination (Module 01, 3.11) — the model fills gaps with its best guess instead of your actual facts.

COMMON MISTAKES

Assuming the model "already knows" company-specific or personal details it was never given. Providing irrelevant context that dilutes the important parts.

BEST PRACTICES

Include only relevant context, clearly delimited from the instruction (Section 3.5). For long context, put the most important parts near the start or end.

SUMMARY

Context grounds the model in your specific situation — the more relevant and precise, the less it has to guess.

3.4 Constraints

DEFINITION

Constraints are explicit limits or rules the output must follow — length, tone, scope, format, or things to avoid.

SIMPLE EXPLANATION

Constraints narrow the space of acceptable answers. Without them, the model picks a reasonable default, which may not match what you actually needed.

💡 Like telling a photographer "no people in the shot, must be landscape orientation, under 10MB" — constraints that shape the deliverable without dictating every artistic choice.

EVERYDAY EXAMPLE

"Write a product description" vs. "Write a product description, under 50 words, no exclamation points, written for a technical audience."

WHY IT MATTERS

Constraints are how you encode business rules, brand voice, and technical limits (like max_tokens, Module 01) directly into the prompt itself.

COMMON MISTAKES

Leaving important constraints unstated and being surprised the model didn't follow them. Stacking too many conflicting constraints at once.

BEST PRACTICES

State constraints explicitly and check for internal conflicts (e.g., "detailed" + "under 20 words" fight each other).

SUMMARY

Constraints are the guardrails that keep output within your actual requirements, not just "a reasonable answer."

3.5 Delimiters

DEFINITION

Delimiters are markers (like XML tags, triple quotes, or dashes) used to clearly separate different sections of a prompt — such as context from instructions.

SIMPLE EXPLANATION

Without delimiters, a long prompt is one undifferentiated wall of text, and the model has to infer where the "document" ends and the "instruction" begins. Delimiters make that boundary explicit.

```
<context>
The customer said the package arrived damaged.
</context>

<task>
Write a polite apology email offering a replacement.
</task>
```

💡 Like using labeled folders instead of dumping every document into one pile — "Invoices" and "Contracts" folders let you (and the model) find the right thing instantly.

EVERYDAY EXAMPLE

Claude responds especially well to XML-style tags like <document>, <instructions>, and <examples> to separate prompt sections.

WHY IT MATTERS

Clear delimiters reduce the model confusing your instructions with content it's supposed to act on — especially important when the "content" itself contains text that looks like instructions.

COMMON MISTAKES

Mixing instructions directly into pasted document text with no visual separation. Using inconsistent delimiter styles within the same prompt.

BEST PRACTICES

Use XML tags for Claude specifically; be consistent; name tags descriptively (`<customer_review>` not `<text1>`).

SUMMARY

Delimiters turn an undifferentiated wall of text into clearly labeled sections the model can parse reliably.

3.6 Role Prompting

DEFINITION

Role prompting means assigning the model a specific professional role or expertise ("You are a senior security auditor") to shape how it approaches the task.

SIMPLE EXPLANATION

Roles activate task-relevant patterns the model learned from training data associated with that role — a "security auditor" role tends to produce more cautious, detail-oriented, risk-focused output than no role at all.

 *Like asking a question to a room of different professionals — a lawyer, a doctor, and a comedian will each answer the same question differently. Assigning a role tells the model which "professional" to channel.*

EVERYDAY EXAMPLE

"You are an experienced code reviewer. Review this function for bugs and security issues." tends to produce more focused, professional feedback than just "review this code."

WHY IT MATTERS

Role prompting is one of the cheapest, most reliable ways to shift the model's tone, depth, and focus without rewriting the whole prompt.

COMMON MISTAKES

Choosing a role too generic to matter ("You are an assistant"). Assigning a role that conflicts with the actual task.

BEST PRACTICES

Choose a role specific enough to actually shift behavior ("senior backend engineer specializing in database performance" beats "engineer").

SUMMARY

Role prompting steers the model's expertise and focus by assigning it a specific professional identity for the task.

3.7 Persona Prompting

DEFINITION

Persona prompting goes beyond role — it defines a consistent character, voice, and personality the model should maintain throughout a conversation, not just a job title.

SIMPLE EXPLANATION

Where role prompting shapes expertise ("what you know"), persona prompting shapes personality and voice ("how you sound") — tone, formality, catchphrases, communication style.

 *Role is the job description on a business card; persona is the person's actual personality, sense of humor, and way of talking — two people with the same job title can have very different personas.*

EVERYDAY EXAMPLE

"You are Aria, a friendly and slightly playful study buddy who explains things with everyday analogies and never sounds condescending" is a persona — it shapes voice across an entire conversation, not just one answer.

WHY IT MATTERS

Consumer-facing AI products (tutoring apps, brand chatbots) live or die on persona consistency — an inconsistent voice breaks user trust and brand identity.

COMMON MISTAKES

Defining a persona once but not reinforcing it in a system prompt that persists across turns, causing "persona drift" in long conversations.

BEST PRACTICES

Put the persona in a system prompt (Module 01, 3.2) so it's reapplied every turn. Keep persona descriptions concrete (traits, tone words, what to avoid) rather than abstract adjectives alone.

SUMMARY

Persona prompting shapes the consistent character and voice of the model across a whole interaction — complementary to, but distinct from, role prompting.

📌 3.8 Output Formatting

DEFINITION

Output formatting is explicitly specifying the structure of the model's response — plain text, bullet list, table, JSON, markdown, a specific template, etc.

SIMPLE EXPLANATION

Without a specified format, the model picks a reasonable default (often prose paragraphs), which may not be what a downstream system or reader actually needs.

💡 Like specifying a file format when you ask a designer for a logo — "send me a .png" versus letting them choose whatever they feel like, when you actually need to import it into specific software.

EVERYDAY EXAMPLE

"List 3 risks" (unspecified format) vs. "Return exactly 3 risks as a JSON array of strings" (specified format, ready for code to consume).

WHY IT MATTERS

Any prompt whose output feeds into code (Module 04, Structured Outputs) absolutely needs explicit output formatting — inconsistent format breaks downstream parsing.

COMMON MISTAKES

Assuming the model will "naturally" return clean JSON without being told to. Not specifying what to do when data is missing or the task can't be completed.

BEST PRACTICES

Show the exact desired format, ideally with a small example. For code-consumed output, explicitly say "return ONLY valid JSON, no extra text."

SUMMARY

Output formatting removes guesswork about how the answer should be shaped — critical whenever a human or a program downstream expects a specific structure.

📌 3.9 Prompt Templates

DEFINITION

A prompt template is a reusable prompt structure with placeholder variables that get filled

in dynamically for repeated, similar tasks.

SIMPLE EXPLANATION

Instead of rewriting a prompt from scratch every time, you write it once with placeholders (like {customer_name} or {document_text}) and swap in new values programmatically.

TEMPLATE:

```
"You are a support agent. Customer: {customer_name}.  
Issue: {issue_description}.  
Write a polite, empathetic reply under {max_words} words."
```

FILLED:

```
"You are a support agent. Customer: Maria.  
Issue: Order arrived damaged.  
Write a polite, empathetic reply under 80 words."
```

💡 *Like a mail-merge template for letters — the structure and tone stay consistent, only the specific details change per recipient.*

EVERYDAY EXAMPLE

A customer support tool that generates a template prompt per ticket, filling in the customer's name, issue, and tone requirements automatically before calling the API.

WHY IT MATTERS

Templates make prompts testable, versionable, and consistent across thousands of requests — essential for any production AI feature (this connects directly to Module 09, Prompt Optimization).

COMMON MISTAKES

Hardcoding values into a "template" instead of using real placeholders, making it not actually reusable. Not validating that all placeholders get filled before sending the request.

BEST PRACTICES

Keep templates in version control, separate from application code. Name placeholders descriptively and validate inputs before substitution.

SUMMARY

Prompt templates turn a one-off prompt into a reusable, maintainable system component — the natural next step once you're sending the same kind of prompt repeatedly.

4. Visual Learning

Anatomy of a Well-Structured Prompt

ROLE / PERSONA	"You are a senior editor..."
CONTEXT	<document>...</document>
INSTRUCTIONS	"Summarize the key risks"
CONSTRAINTS	"Under 100 words, no jargon"
OUTPUT FORMAT	"As a bulleted list"

Role vs. Persona

Prompt Identity Shaping	
└─ Role Prompting	(what expertise to apply)
└─ "You are a data scientist"	
└─ Persona Prompting	(what voice/personality to use)
└─ "Friendly, upbeat, uses simple analogies"	

Prompt Component Comparison

Component	Answers the question	Example
Instructions	What should I do?	"Summarize this document"
Context	What background do I need?	The document text itself
Constraints	What are the limits?	"Under 100 words"
Delimiters	Where does each part start/end?	XML tags, triple quotes
Role	What expertise should I use?	"You are a lawyer"
Persona	What voice should I use?	"Friendly and casual"
Output Format	How should the answer look?	"As a JSON array"

5. Real World Applications

Product	How these fundamentals show up
Claude / ChatGPT / Gemini system prompts	Combine persona, role, and constraints to keep the assistant consistent, safe, and on-brand across millions of conversations.
GitHub Copilot / Cursor / Claude Code	Uses heavy context (open files, repo structure) and tight output-format instructions (valid code only, specific diff format).
Customer support AI	Role + persona + constraints combine to keep replies on-brand, empathetic, and policy-compliant, using prompt templates per ticket type.
Document analysis tools	Heavy use of delimiters to separate the uploaded document from the analysis instructions.

6. Practical Examples

Beginner: Instruction vs. no instruction

Ask an LLM "Paris" with nothing else, then ask "Give me 3 must-see landmarks in Paris for a first-time visitor, each in one sentence." Compare how much more useful the second response is.

Beginner: Adding constraints

Ask for "a product description," then ask again with "under 30 words, no adjectives, written for a technical audience." Notice how much the constraints change the result.

Intermediate: Role prompting shift

Ask the same question ("Should I use a monolith or microservices for a small startup?") with no role, then with "You are a pragmatic staff engineer who values simplicity over trends." Compare tone and content.

Advanced: Full structured prompt

Write one prompt combining role + context (a pasted paragraph) + instructions + constraints + output format (JSON), following the anatomy diagram in Section 4. Test it against an unstructured version of the same ask.

7. Code Examples

🐍Python — Structured prompt with delimiters and role

```
# pip install anthropic
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

system_prompt = (
    "You are a senior customer support agent. " # role
    "Always be polite, empathetic, and concise." # persona traits
)

user_prompt = """
<context>
Customer: Maria. Order #4521 arrived damaged.
</context>

<task>
Write a reply offering a replacement.
</task>

<constraints>
Under 80 words. No exclamation points.
</constraints>
"""

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=200,
    system=system_prompt,
    messages=[{"role": "user", "content": user_prompt}]
)

print(response.content[0].text)
```

📄JavaScript — Prompt template with placeholder filling

```
// npm install @anthropic-ai/sdk
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

// A reusable template with placeholders
function buildPrompt({ customerName, issue, maxWords }) {
    return `You are a support agent. Customer: ${customerName}.
    Issue: ${issue}.
    Write a polite, empathetic reply under ${maxWords} words.`;
}

const prompt = buildPrompt({
    customerName: "Maria",
    issue: "Order arrived damaged",
    maxWords: 80,
});

const response = await client.messages.create({
    model: "claude-sonnet-4-6",
    max_tokens: 200,
    messages: [{ role: "user", content: prompt }],
});

console.log(response.content[0].text);
```



Tip

Keep your system prompt (role/persona/global constraints) separate from your user prompt (specific task/context) — it makes templates far easier to reuse and test.

8. Common Mistakes

- **Writing vague instructions** and expecting the model to infer your exact intent.
- **Omitting relevant context** and being surprised the model "doesn't know" your specific situation.
- **Leaving constraints unstated** — length, tone, and scope should be explicit, not assumed.
- **Skipping delimiters** on long prompts, causing the model to confuse content with instructions.
- **Choosing a generic role** that doesn't actually shift behavior ("You are an assistant" adds little).
- **Defining persona once** without reinforcing it in a persistent system prompt, causing drift in long conversations.
- **Not specifying output format** for anything that feeds into downstream code.

9. Best Practices

- **Structure prompts consistently:** role → context → instructions → constraints → format.
- **Use XML-style delimiters** for Claude specifically to clearly separate prompt sections.
- **Be specific with roles** — narrow expertise beats generic titles.
- **Put persona in the system prompt** so it persists across every turn of a conversation.
- **Always specify output format explicitly** when the answer feeds into code or a strict process.
- **Turn repeated prompts into templates** with real placeholders, kept in version control.

10. Hands-on Activities

Do these using any LLM chat interface or API playground you have access to.

Easy 5 exercises

1. Take a vague one-line prompt you'd normally write and rewrite it with an explicit instruction, one constraint, and a specified format.
2. Write the same request with and without a role ("You are a nutritionist") and compare the two responses.
3. Wrap a block of pasted text in `<document>` tags and ask the model to summarize only what's inside the tags.
4. Write a persona description (3-4 traits) for a study-buddy chatbot and test it across 3 different questions for consistency.
5. Ask for output as a markdown table, then ask the same question with no format specified — compare.

Intermediate 3 exercises

1. Build a full structured prompt (role + context + instructions + constraints + format) for a real task you actually need done.
2. Turn that prompt into a template with at least 2 placeholders, and manually "fill" it for 2 different inputs.
3. Deliberately write a prompt with two conflicting constraints (e.g., "very detailed" + "under 15 words") and observe how the model resolves the conflict.

Challenge 2 exercises

1. Design a system prompt for a fictional AI product (your choice) combining role, persona, and 3 global constraints — write a short justification for each choice.
2. Take a real messy, unstructured prompt (yours or found online) and refactor it into the 5-part anatomy from Section 4. Compare outputs before and after.

11. Mini Project

Goal

Build a small "prompt template engine" script: a reusable function that fills in a template and calls Claude, for a task of your choice (e.g., generating personalized emails).

Requirements

- An Anthropic API key
- Python or JavaScript with the Anthropic SDK
- A template string with at least 3 placeholders

Step-by-Step Instructions

1. Write a template function that takes a dictionary/object of values and returns a filled prompt string (see Section 7 example).
2. Define a system prompt with a role and persona relevant to your task.
3. Add explicit constraints and an output format to the template.
4. Call the API with 3 different sets of input values and print all 3 results.
5. Check: did each output respect the constraints and format every time?

Expected Output

```
=== Input Set 1 ===
[filled prompt shown]
[model output]

=== Input Set 2 ===
[filled prompt shown]
[model output]

=== Input Set 3 ===
[filled prompt shown]
[model output]
```

Possible Improvements

- Add input validation to catch missing placeholder values before calling the API.
- Store templates in separate files, loaded at runtime.
- Add automatic output-format validation (e.g., confirm valid JSON) with a retry on failure.

12. Review Questions

Multiple Choice (10)

1. What does "context" provide in a prompt?

- A) The output format B) Background information the model needs C) The model's temperature D) A random seed

2. Constraints in a prompt primarily do what?

- A) Add background facts B) Limit/shape the acceptable output C) Assign a persona D) Increase token count automatically

3. What are delimiters used for?

- A) Making text bold B) Separating sections of a prompt clearly C) Reducing token cost D) Setting temperature

4. Role prompting primarily shapes:

- A) The model's expertise/approach B) The API pricing C) The context window size D) The tokenizer used

5. Persona prompting primarily shapes:

- A) Max tokens B) Voice, tone, and personality C) The programming language used D) The context window

6. Why specify output formatting explicitly?

- A) It's required by law B) It removes guesswork, especially for code-consumed output C) It lowers temperature automatically D) It's optional and rarely useful

7. A prompt template is best described as:

- A) A one-time-use prompt B) A reusable prompt with placeholders for repeated tasks C) A type of delimiter D) A system-only feature

8. Where should persona be defined for consistency across turns?

- A) In every user message manually B) In the system prompt C) It can't persist across turns D) In the max_tokens parameter

9. Which is a good example of a specific, effective instruction?

- A) "Do something with this" B) "Summarize the payment terms in 3 bullet points" C) "Text" D) "Maybe explain if you want"

10. What commonly happens when constraints conflict (e.g., "detailed" + "under 15 words")?

- A) The API rejects the request B) The model must resolve the tension, often imperfectly C) Temperature auto-adjusts D) Nothing, constraints never conflict

Identification (5)

1. _____ is the explicit statement of what action the model should take.

2. _____ are markers used to separate distinct sections of a prompt.

3. _____ assigns the model a professional expertise to shape its approach.

4. _____ defines a consistent character, voice, and personality across a conversation.

5. _____ is a reusable prompt structure with placeholder variables.

True or False (5)

1. Role prompting and persona prompting mean exactly the same thing.

2. Delimiters help the model avoid confusing pasted content with instructions.

3. Leaving output format unspecified is fine for prompts whose output feeds directly into code.

4. Constraints can be about length, tone, scope, or things to avoid.

5. A prompt template with hardcoded values instead of real placeholders is not actually reusable.

Situational (5)

1. Your AI feature's output needs to be parsed by code downstream, but responses come back as inconsistent free text. What's missing from the prompt?

2. A long chatbot conversation starts sounding inconsistent in tone after 20 turns. What persona-related fix would you apply?

3. You paste a long document directly before your instructions with no separation, and the model gets confused about what to summarize vs. what to do. What's the fix?
4. You're sending a similar prompt hundreds of times a day with only the customer name and issue changing. What should you build instead of copy-pasting each time?
5. Your prompt says "write a thorough analysis" and also "keep it to 2 sentences." What's wrong, and how would you fix it?

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-A, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-Instructions, 2-Delimiters, 3-Role prompting, 4-Persona prompting, 5-Prompt template

True or False: 1-False, 2-True, 3-False, 4-True, 5-True

Situational (sample reasoning):

1. Missing explicit output formatting — specify the exact structure (e.g., "return ONLY valid JSON").
2. Reinforce the persona in the system prompt so it's reapplied every turn instead of drifting.
3. Use delimiters (e.g., XML tags) to clearly separate the document from the instructions.
4. A prompt template with placeholders for the variable fields (customer name, issue).
5. Conflicting constraints ("thorough" vs. "2 sentences") — resolve by choosing one priority or restructuring (e.g., "a thorough analysis, summarized into exactly 2 sentences").

13. Cheat Sheet

Prompt Anatomy

Role/Persona → Context → Instructions → Constraints → Output Format. Not every prompt needs all five, but know what each does.

Delimiters

Use XML tags to separate sections: <context>, <task>, <constraints>, <examples>.

Role vs. Persona

Role — what expertise to apply.

Persona — what voice/personality to use.

Both belong in the system prompt.

Templates

Real placeholders, not hardcoded values. Validate before filling. Keep in version control.

14. Key Takeaways

- Most bad AI outputs trace back to a prompt problem — vague instructions, missing context, or unstated constraints.
- Instructions state the action; context provides background; constraints set limits; delimiters mark boundaries.
- Role prompting shapes expertise; persona prompting shapes voice and personality — both belong in the system prompt for consistency.
- Explicit output formatting is essential whenever the response feeds into code or a strict downstream process.
- Prompt templates turn repeated one-off prompts into reusable, testable, versionable system components.

15. Additional Resources

Official Documentation

- Anthropic — Prompt engineering overview: docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- Anthropic — Use XML tags to structure prompts (Claude docs)
- OpenAI — Prompt engineering guide: platform.openai.com/docs

Tools

- Anthropic Console — prompt testing playground with system/user message separation.
- Prompt template managers (e.g., LangChain's PromptTemplate) for larger production systems.

Communities & Further Learning

- Anthropic's prompt library for example structured prompts across common use cases.
- Provider changelogs — prompting best practices evolve as models improve; check current docs periodically.